Error Handling Advanced

CS-3180

Error Handling

- When things go wrong, we used `Result<T>`
- Very flexible option
- We'll look at some other Options (pun intended)
- As well as how to integrate them into the type system

Options

- `Result` is used everywhere in the standard library
- With a mix of types for the generics
- But often things don't "go wrong"
- They are simply "missing"

```
1 enum Result<T, E> {
2     Ok(T),
3     Err(E),
4 }
```

Null

- Almost all languages handle a missing value with `null` / `nil`
- `null` is often credited to Tony Hoare (QuickSort Invented)
- "Because it was so easy to implement!"
- Added null references to ALGOL in 1965
- Later called it the "Billion Dollar Mistake"

Rust and Null

- Rust simply does not have the concept of null
- `null` is almost always a runtime error
- `NullPointerException` extends `RuntimeException` in Java
- In Rust, if the data is invalid, the compiler will ensure it doesn't run
- Rust solves this problem with the Option type

Option

- This definition is *everything* you need to not have null
- `Option` is a generic enum
- `Option::None` is a variant, but not a invalid reference
- `Option::Some` variant that wraps the data

```
pub enum Option<T> {
    None,
    Some(T),
}
```

Option

- Let's re-examine our divide function from before
- Use `Some(...)` to return `Option::Some` variant
- Use `None` to return `Option::None`

```
1 fn divide(num: f64, demon: f64) -> Option<f64> {
2     if demon == 0.0 {
3         None
4     } else {
5         Some(num / demon)
6     }
7 }
```

Option vs. Result

- Accomplish similar goals
- Use whichever is more idiomatic to your use case
- I tend to reach for `Result` more than `Option`
- Let's look at how they both fit into Rust error handling system a little deeper

Domain-Specific Errors

- Before, we had a lot `Result` whose error variant held a string
- The standard library often gives a more *specific* error type

```
let path: &str = "file.txt";
// read a file content into a string
let result: Result<String, std::io::Error>
    = fs::read_to_string(path);
```

Multiple Error Types

- If we wanted to use the ? operator
- We would need a consistent type to return
- What type could we even put here?

```rust
fn read_and_parse_number(path: &str) -> Result<i32, ???> {
    // wants to return a std::io::Error
    let contents = fs::read_to_string(path)?;

    // wants to return a std::num::ParseIntError
    let number: i32 = contents.trim().parse()?;

    Ok(number + 10)
}
```

Stack Problems

- Rust needs to know how much space to allocate for your data
- Different types might have different sizes
- A solution to this starts by allocating data on the heap
- `Box<T>` special type that puts T on the heap
- Returns a *constant-sized* pointer to that data

Heap Allocations

- Heap Allocation on its own
- Not particularly useful
- Rust will mostly coerce pointers for you

```
1 let a: Box<i32> = Box::new(45);
2
3 // use deref if needed, Box will do this for you
4 println!("Value of a was: {}", *a);
5 println!("Value of a was: {}", a);
```

Dynamic Dispatch

- Almost all error types are secretly implementing an `Error` trait
- (Traits are similar to interfaces, more on this later)
- But `std::error::Error` is a uniform type we can return
- Need to use the `dyn` keyword to opt-in to dynamic dispatch

Box + Dyn

- Revisit that previous example
- Use both `Box` and `dyn`
- (Typically you won't qualify all of these types)

```
fn read_and_parse_number(path: &str)
        -> Result<i32, Box<dyn std::error::Error>> {
    // can return Error
    let contents = fs::read_to_string(path)?;

    // can return Error
    let number: i32 = contents.trim().parse()?;

    Ok(number + 10)
}
```

Concise Control Flow

- You are always welcome to `match` Result and Option
- Can get very verbose very quickly
- Rust provides special syntax to handle things more concisely
- `if let` and `let else`

If Let

- Often, you only care about the "success" case
- `Some` or `Ok` match
- If you always matched, you would often get:

```
let optional = Some(7);

match optional {
    Some(i) => println!("A long string and {:?}", i),
    _ => {},
};
```

If Let

- `if let Some(bind) = value`
- Destructures value into binding within the scope

```
let number = Some(7);
if let Some(i) = number {
    println!("Matched {:?}", i);
}

let letter: Option<i32> = None;

// use else to specify failure case
if let Some(i) = letter {
    println!("Matched {:?}", i);
} else {
    println!("Didn't match a number");
}
```

Let else

- Often, we want to early return from a function
- Common in other error handling systems
- Use `let Ok(bind) = expression else {...}`
- Works with both `Ok(...)` + `Some(...)`
- Let's see that:

Let Else

- Syntax almost feels backwards
- Commonly used with `Option`

```rust
fn parse_and_print(s: Option<&str>) -> i32 {
    let Some(parse_me) = s else {
        println!("None provided");
        return 0;
    };

    if let Ok(value) = parse_me.parse() {
        println!("{value}");
        return value;
    }
    // probably don't want something like this
    0
}
```

Let Else

- Syntax almost feels backwards
- Commonly used with `Option`

```rust
fn parse_and_print(s: Option<&str>) -> i32 {
    let Some(parse_me) = s else {
        println!("None provided");
        return 0;
    };

    if let Ok(value) = parse_me.parse() {
        println!("{value}");
        return value;
    }
    // probably don't want something like this
    0
}
```

Let Else

- Syntax almost feels backwards
- Commonly used with `Option`

```rust
fn parse_and_print(s: Option<&str>) -> i32 {
    let Some(parse_me) = s else {
        println!("None provided");
        return 0;
    };

    if let Ok(value) = parse_me.parse() {
        println!("{value}");
        return value;
    }
    // probably don't want something like this
    0
}
```

Conversions

- If a value is missing
- There might not be a sensible default to match to
- But `?` only works for `Result`
- Very common to want to convert from `Option` to `Result`
- Few methods, but I'll show you the most common

Ok Or

- Function on `Option` that transforms it into a result
- `Some(...)` -> `Ok(...)`
- You provide the error variant

```
let maybe_value: Option<i32> = Some(42);

let result = maybe_value.ok_or("Value was missing");

match result {
    Ok(num) => println!("Matched: {num}"),
    Err(err) => println!("Error: {err}")
}
```

Unwrap Or

- Both `Result` and `Option` provide an `or_else` method
- "Give me the value, or else..."

```
let some_val: Option<&str> = Some("Rust");
let none_val: Option<&str> = None;

println!("{}", some_val.unwrap_or("Default"));
println!("{}", none_val.unwrap_or("Default"));
```

Summary

- Rust provides `Option` for optional values
- No concept of null
- `Box<dyn Error>` for dynamic error handling
- `if let` and `let else` for conciseness
- Convert between types explicitly

Questions?